

1. Transaction Concepts & Lifecycle

Logical Unit of Work

Logical unit of work executing database read/write ops maintaining integrity.

- Active → Partially Committed → Committed
- Active/Partially Committed → Failed → Aborted → Terminated
- **Active:** Initial state while executing instructions.
 - **Partially Committed:** Final instruction executed; writes still in memory buffer.
 - **Committed:** Log flushed to disk; success ack sent.
 - **Failed:** Hardware/logical error occurs.
 - **Aborted:** Changes undone (rollback); restart or terminate.

2. ACID Properties Summary Matrix

Core Engine

Property	Meaning	Responsible DBMS Component
Atomicity	All operations execute or none do ("All or Nothing")	Recovery Manager (Undo Logs / WAL)
Consistency	Database transforms from one valid state to another	Application Programmer + Integrity Constraints
Isolation	Concurrent transactions run as if serial (no interference)	Concurrency Control Manager (Locks/Timestamps)
Durability	Committed changes survive any subsequent crash	Recovery Manager (Redo Logs / Stable Storage)

Key Rule: Intermediate uncommitted changes must never be visible to other transactions (Preserves Isolation).

3. Concurrency Anomalies & Isolation Violations

AKTU 10M Question

Anomaly	Conflict Pattern	Description
Dirty Read (WR)	$W1(A) \rightarrow R2(A) \rightarrow A1$	T2 uncommitted data read karta hai jo T1 abort hone par discard ho jata hai.
Lost Update (WW)	$R1(A) \dots R2(A) \dots W1(A) \dots W2(A)$	T2 T1 ke update ko bina jane overwrite kar deta hai (Blind/unaware write).

Anomaly	Conflict Pattern	Description
Unrepeatable Read (RW)	$R1(A) \rightarrow W2(A) \rightarrow R1(A)$	T1 do baar read karta hai par intermediate update ke karan value badal jati hai.
Phantom Read	$R1(\text{range}) \rightarrow \text{Ins2} \rightarrow R1(\text{range})$	Predicate query me new tuples insert hone se record count change hota hai.

4. Schedules & Serializability Foundations

Equivalence Base

- **Serial Schedule:** Transactions execute strictly one after another. Concurrency = 0, Throughput = Minimum, Consistency = 100%.
- **Non-Serial Schedule:** Operations interleave. High throughput, but anomaly prone.
- **Serializable Schedule:** A non-serial schedule equivalent to some serial schedule.
- **Conflict Operations:** Belong to T_i e T_j , access same data item Q , at least one is Write. Conflicts: $(R_i, W_j), (W_i, R_j), (W_i, W_j)$. Read-Read (R_i, R_j) is NON-conflicting.

5. Conflict Serializability & Precedence Graph

Graph Algorithm

Conflict Equivalent: $SS \equiv_C S'S$ if SS transformed into $S'S$ by swapping non-conflicting adjacent operations.

Precedence Graph $G = (V, E)$: Node per transaction T_i .

Directed Edge $T_i \rightarrow T_j$ iff T_i executes conflicting op before T_j .

AKTU Theorem: Schedule S is Conflict Serializable **IF & ONLY IF** Precedence Graph $G(S)$ contains **NO CYCLES (Acyclic)**.

Equivalent Serial Order: Derived directly using **Topological Sort** on the acyclic DAG.

6. View Serializability (VSR) & Blind Writes

NP-Complete Testing

Schedule $S \equiv_V S'$ (View Equivalent) iff 3 conditions hold:

- Initial Read:** Agar T_i $\$S\$$ me initial XX read karta hai, to S' me bhi wahi kare.
- Updated Read:** Agar T_j $\$S\$$ me T_i ka likha XX read karta hai, to S' me bhi kare.
- Final Write:** Jo transaction $\$S\$$ me final write karta hai, wahi S' me kare.

Blind Write Shortcut Rule:

- Blind Write: Transaction writes XX without reading XX first ($SW(X)$ without $RR(X)$).
- Agar schedule me **NO blind write** hai aur schedule **CSR nahi hai**, then schedule can **NEVER** be View Serializable!

Conflict Serializable (CSR) $\subset$ View Serializable (VSR)

7. Recoverability Spectrum & Cascading Aborts

Critical for Durability

Schedule Class	Formal Condition	Cascading Rollback?	AKTU Notes
Irrecoverable	T_j reads uncommitted from T_i , and $Commit(T_j) < Commit(T_i)$	Catastrophic!	Strictly disallowed in DBMS (Violates Durability).
Recoverable	$Commit(T_i) < Commit(T_j)$ whenever T_j reads uncommitted from T_i	Possible	Agar T_i abort hua to T_j bhi rollback hoga (Cascading Abort).
Cascadeless (ACA)	$Commit(T_i) < Read_j(X)$ (No uncommitted read allowed)	Zero (None)	Reads only committed data. High concurrency.
Strict	$Commit/Abort(T_i) < Read_j(X) \text{ ext}\{ \text{ and } \} Write_j(X)$	Zero (None)	Undo easily by restoring Before-Image (Old Value).

Hierarchy: Strict Schedules $\subset$ Cascadeless Schedules $\subset$ Recoverable Schedules $\subset$ All Schedules

8. Two-Phase Locking (2PL) Protocols

Serializability Guard

Locks: Shared $S(Q)$ (read-only, shared), Exclusive $X(Q)$ (read/write, exclusive).

- Growing Phase:** Transaction acquires locks, cannot release any.
- Shrinking Phase:** Transaction releases locks, cannot acquire any.
- Lock Point:** Point where the transaction acquires its final lock.

Variant	Lock Holding Rule	Guarantees
Basic 2PL	Normal 2 phases	CSR, Deadlock & Cascading possible
Strict 2PL	X-locks held till Commit	CSR + Cascadeless + Strict
Rigorous 2PL	S & X locks held till Commit	CSR + Commit Order Serializability
Conservative	Acquire all locks at start	Deadlock-Free (Low concurrency)

9. Deadlock Handling & Timestamp Schemes

Wait-Die vs Wound-Wait

Timestamp Scheme: $TS(T_i)$ timestamp at start. Smaller TS = Older.

Protocol	Rule when T_i requests lock held by T_j	Type
Wait-Die	Older waits for Younger ($TS_i < TS_j \implies \text{ext}\{\text{Wait}\}$). Younger dies ($TS_i > TS_j \implies \text{ext}\{\text{Die/Rollback}\}$).	Non-preemptive
Wound-Wait	Older wounds/preempts Younger ($TS_i < TS_j \implies \text{ext}\{\text{Rollback } T_j\}$). Younger waits ($TS_i > TS_j \implies \text{ext}\{\text{Wait}\}$).	Preemptive (Starvation Free)

Deadlock Detection: Wait-For Graph (WFG) directed edges $T_i \rightarrow T_j$. Cycle $\iff$ Deadlock. Victim selection based on minimum rollback cost.

10. Database Failures & WAL Protocol

Recovery Foundation

- Storage Hierarchy:** Volatile (RAM), Non-Volatile (Disk/SSD), Stable (Mirrored disks, survives crashes).
- Failure Types:** Transaction Failure (logical/system), System Crash (RAM loss), Disk Failure (head crash).
- Write-Ahead Logging (WAL):** Log record $\langle T_i, X, V_{old}, V_{new} \rangle$ must be written to stable storage BEFORE database item X is updated on disk.

Technique	Disk Write Timing	Recovery Action
Deferred Mod	Only after Commit	REDO only (No UNDO required)
Immediate Mod	Anytime (Uncommitted)	Both UNDO & REDO required

11. Checkpoints & Shadow Paging

Performance Boost

Checkpoints: Limits log scan overhead on system crash.

- Log Record: $\langle \text{checkpoint } L \rangle$ where L = active transactions list.
- Scan backward to last checkpoint.
 - T_i with Commit/Abort $\rightarrow$ REDO list (Forward scan).
 - T_i active without Commit $\rightarrow$ UNDO list (Backward scan).

Shadow Paging: Maintains Current Page Table in RAM, Shadow Page Table on disk. Commit = atomically pointing to current page table. No logs needed, but page fragmentation occurs.

12. Distributed Databases (DDBMS) & Directory Systems

Architecture & Protocols

Data Fragmentation:

- Horizontal:** Rows split by predicate $\sigma_p(R)$. Reconstruct: $R_1 \cup R_2$.
- Vertical:** Columns split by projection $\pi(R)$ keeping PK. Reconstruct: $R_1 \bowtie R_2$.
- Hybrid:** Nested horizontal & vertical.

Correctness Rules:

- Completeness:** Har tuple/attribute fragment me hona chahiye.
- Reconstruction:** Lossless join/union se original relation restore hona chahiye.
- Disjointness:** Non-key attributes overlap nahi hone chahiye.

Directory & 2PC Protocol:

- LDAP:** Lightweight Directory Access Protocol (X.500 based, DIT tree, read-optimized).
- Two-Phase Commit (2PC):**
Phase 1: Prepare & Vote (Commit/Abort).
Phase 2: Global Decision (Commit/Abort).

13. Master AKTU Exam Problem Solving Flowchart & Checklist

100% Exam Formulae

Step	Question Type	Actionable Solution Algorithm	Outcome / Formula
1	Conflict Serializability (CSR)	Identify conflicting pairs $(R_i, W_j), (W_i, R_j), (W_i, W_j)$ on same variable. Draw edge $T_i \rightarrow T_j$.	Acyclic $\implies$ CSR. Cyclic $\implies$ Not CSR. Topological sort = Serial order.
2	View Serializability (VSR)	Agar CSR hai $\implies$ Directly VSR! Agar Not CSR hai, check for Blind Writes.	Agar koi blind write nahi hai $\implies$ Directly NOT VSR! If blind write present, test equivalence.
3	Recoverability Check	Locate every Read-From pair: $W_i(X) \dots R_j(X)$ where T_j reads uncommitted X from T_i .	If $C_i < C_j \implies$ Recoverable. If $C_j < C_i \implies$ Irrecoverable. If $C_i < R_j(X) \implies$ Cascadeless.
4	2PL Schedule Verification	Trace every lock acquisition and release. Check if any lock acquired AFTER a release.	If lock acquired after unlock $\implies$ Violates 2PL. If all X-locks held till commit $\implies$ Strict 2PL.
5	Log Recovery (Undo/Redo)	Find checkpoint. Transactions with 'commit' record $\rightarrow$ 'REDO'. Transactions without 'commit' $\rightarrow$ 'UNDO'.	Execute backward for 'UNDO' (restore V_{old}) and forward for 'REDO' (apply V_{new}).